Module 5

Reinforcement Learning Policy Iterations and Value Iterations

Solving the Hamilton-Jacobi-Bellman (HJB) equation and finding the optimal control action in real time is difficult in general. Bellman recommended in [1]-[2] to use of the successive approximation techniques, based on either *guessing an approximate initial value of the performance criterion (function) and performing corresponding iterations (iterations in the function space) or by guessing an approximate initial value of the optimal control (policy) and running corresponding iterations in the policy space*. These techniques, respectively also known as the *value iterations* (VIs) and the *policy iterations* (PIs) can converge either to the optimal performance value or to the optimal policy (optimal control strategy) or to the both optimal performance and optimal policy.

For the successive approximation technique in continuous time, Bellman established monotonicity of PIs for the scalar case in [1]-[2]. In [3]-[4], the successive approximations for solving the continuous-time dynamic programming problem were further researched, and an extension to the discrete-time dynamic programming successive approximations problem was presented. Bellman favored the PI iterations over the VI iterations since in [3], he stated: "Let us think with the observation that dynamic programming processes generate two types of functions, return functions and policy functions. As far as the actual decision process is concerned, it is far more natural to think in terms of an approximate *policy* rather than in terms of an approximate *function*". Bellman showed monotonic convergence for PIs, in both [2] and [3] books, and stated that VIs may converge quite slowly, but not monotonically.

The VIs and PIs were further researched, improved, clarified, and generalized over the next fifty-sixty years with most noticeable results reported in the following publications: [5]-[7] in continuous-time, [8]-[9] in discrete-time, [10]-[11] for affine nonlinear systems, in the extraordinary book [12], the breakthrough paper by Murray and his coworkers, [13], in [14] for discrete-time, and in [15] for the stochastic case.

The book [12] presents the value iterations method in a stochastic setup with the use of neural networks for function approximations. The book [16] presents PIs and VIs as approximate dynamic programming techniques in the framework of reinforcement learning (RL). Book [17] discusses the VIs and PIs mostly in the stochastic setup with emphasis on VIs.

This module is organized as follows. Section 5.1 present continuous-time successive approximations of [5]-[6] for general nonlinear time-invariant systems. In Section 5.2, the discrete-time successive approximations deterministic algorithms of [8]-[9] are presented as RL PIs algorithms. Sections 5.3 and 5.4 consider the VIs methods in discrete-time. PIs and VIs are well covered in both continuous- and discrete-time domains in several overview and tutorial papers [18]-[21].

5.1 Continuous-Time Iterations in Policy Space

Iterations in policy space (successive approximations) for continuous-time systems for general nonlinear systems were systematically considered for the first time in [5]-[6]. The requirements for obtaining analytical solutions for approximate controls and finding analytically the performance criterion gradient in each iteration has had considerably limited applicability of that algorithm to modern approaches of RL. However, it can be used in one very important application, the linear-quadratic (LQ) optimization case.

5.1.1 Vaisbord-Milhstein's Policy Iterations Algorithm

The main result of [5] is a successive approximation algorithm, which in fact is the PIs algorithm of RL. Almost the identical algorithm was reported in [6]. Consider a time invariant dynamic optimization problem defined for a controlled nonlinear system defined by

$$\frac{dx(\tau)}{d\tau} = f(x(\tau), u(\tau)), \qquad x(t) = x_0 \tag{5.1}$$

where $f(x(\tau), u(\tau))$ is a continuously differentiable vector function, and t is an arbitrary variable initial time instant. The goal is to minimize the performance criterion (functional) using feedback control $u(x(\tau))$, and find the optimal (minimal) value of the performance criterion

$$J^{\text{opt}}\left(x(t)\right) = \min_{u(x(\tau)),\ t\le\sigma\le\infty} \left\{ \int_t^{\infty} g(x(\sigma), u(\sigma))d\sigma \right\} \tag{5.2}$$

where $g(x(\sigma), u(\sigma))$ is a nonnegative continuously differentiable scalar function.

Algorithm of Vaisbord-Milhstein

Step 0: This policy iteration algorithm starts with some initial admissible (stabilizing) approximate optimal feedback control $u_1(x(\tau))$ and generates a sequence of approximate controls $u_2(x(\tau)),...,u_k(x(\tau)),u_{k+1}(x(\tau))$ via the following steps.

Step 1: Solve analytically

$$\frac{dx_k(\tau)}{dt} = f(x_k(\tau), u_k(\tau)), \qquad x_k(t) = x_0, \quad k = 1,2,3,... \tag{5.3}$$

Step 2: Evaluate analytically the approximate performance

$$J_k\left(x(t)\right) = \int_t^{\infty} g(x_k(\sigma), u(x_k(\sigma)))\, d\sigma \tag{5.4}$$

and find analytically the gradient vector $\left(\frac{\partial J_k(x(t))}{\partial x}\right)$.

Step 3: Determine analytically the approximate control vector $u_{k+1}(x(\tau))$ that minimizes

$$\min_{u_{k+1}(x)} \left\{ g(x, u_{k+1}) + \left(\frac{\partial J_k}{\partial x}\right)^T f(x, u_{k+1}) \right\} \tag{5.5}$$

by solving analytically the obtained algebraic equation with respect to $u_{k+1}(x(\tau))$. Note that the gradient of the performance criterion in (5.5) is fixed in the previous iteration. Assume that the obtained algebraic equation, after application of the min operator, has a unique solution with respect to the approximate control vector $u_{k+1}(x(\tau))$, that is

$$\frac{\partial}{\partial u_{k+1}} \left\{ g(x, u_{k+1}) + \left(\frac{\partial J_k}{\partial x}\right)^{\mathrm{T}} f(x, u_{k+1}) \right\} = 0 \quad \Rightarrow \quad u_{k+1}(x) \tag{5.6}$$

Step 4: The obtained sequence of the approximate performance criteria monotonically converges to the value of the optimal performance criterion, that is, the papers showed

$$J_1(x(t)) \geq J_2(x(t)) \geq \cdots \geq J_k(x(t)) \geq J_{k+1}(x(t)) \tag{5.7}$$

Repeat Steps 1-4 until the desired accuracy is obtained.

It is obvious that due to the requirements of obtaining analytical solutions in several instances, this algorithm has a quite limited applicability. However, it can be used in one very important application, the LQ optimization case. The solution of the LQ optimization problem via the Vaisbord-Milhstein algorithm was presented in Module 4, known also in the modern literature as Kleinman's algorithm [22].

5.2 Policy Iterations in Discrete Time

As indicated in [1]-[2], the PIs require an initial guess with respect the feedback policy, that is, an initial guess is made for $u^{(0)}(x(k))$. With that initial guess, an approximate performance criterion is evaluated iteratively from by running several iterations as presented in Larson [8]-[9]

$$\begin{aligned} J^{opt^{(0,j+1)}}(x(k)) &= g\left(x(k),u^{(0)}(x(k))\right)+J^{opt^{(0,j)}}\left(f\left(x(k),u^{(0)}(x(k))\right)\right) \\ J^{opt^{(0,0)}}(x(k)) &= 0, \quad j=0,1,2,... \end{aligned} \tag{5.8}$$

These iterations produce $J^{opt^{(0,j+1)}}(x(k))$. The updated optimal control approximation is evaluated from the Bellman functional equation

$$\begin{aligned} &\min_{u(k)}\left\{g(x(k),u(k))+J^{opt^{(0,j+1)}}\left(x(k+1)\right)\right\} \\ &\Rightarrow \frac{\partial}{\partial u(k)}\left\{g(x(k),u(k))+J^{opt^{(0,j+1)}}\left(x(k+1)\right)\right\}=0 \Rightarrow u^{(1)}(x(k)) \end{aligned} \tag{5.9}$$

which generates $u^{(1)}(x(k))$. The iterative procedure (5.8) is repeated with $u^{(1)}(x(k))$ to generate $J^{opt^{(1,1)}}(x(k))$. In general, the policy iterations include two iterative schemes: the inner loop is with respect to the performance value iterations and the outer loop is with respect to the optimal policy iterations. Together, the both loops can be represented in one iterative formula known as the policy iterations.

Policy Iteration Algorithm:

$$\begin{aligned} J^{opt^{(i,j+1)}}(x(k)) &= \min_{u(k)}\left\{g\left(x(k),u^{(i)}(k)\right)+J^{opt^{(i,j)}}\left(f\left(x(k),u^{(i)}(k)\right)\right)\right\} \\ J^{opt^{(i,0)}}(x(k)) &= 0, \quad j=0,1,2,3,... \quad i=0,1,2,3,... \end{aligned} \tag{5.10}$$

Note that for each $u^{(i)}(x(k)), i=1,2,3,...$ obtained via the minimization process several iterations have to be run for $J^{opt^{(i,j+1)}}(x(k))$ until it converges starting each time with the zeroth initial condition $J^{opt^{(i,0)}}(x(k))=0$.

Comment 5.1: Note that *j*-iterations do not need to be run until full convergence with very high accuracy. It is sufficient for the given policy to get a satisfactory accuracy for the approximate optimal performance, and then perform the minimization and go to the next approximate optimal policy. This has been the case also in the generalized policy iterations of the temporal differences algorithms from Chapter 6 of [23].

5.2.1. Policy Iterations for Discrete-Time LQ Optimal Control

In Module 3, we have introduced the classic *policy iterations* (successive approximations) algorithm of Hewer [24]-[25] for solving the time-invariant LQ problem. For the reason of completeness, the Hewer algorithm will be briefly reviewed here. Policy iterations are developed for the steady state LQ optimal control problem when the optimization period theoretically goes to infinity and the optimal feedback gain becomes constant. Consider a linear system with a quadratic performance criterion

$$x(k+1) = Ax(k) + Bu(k), \quad x(k) = \text{given initial condition} \tag{5.11}$$

$$\begin{aligned} J(x(k)) &= \frac{1}{2}\sum_{k}^{\infty}\left[x^T(k)R_1x(k) + u^T(k)R_2u(k) \right] = \frac{1}{2}\sum_{k}^{\infty} r(k) \\ &= \frac{1}{2}(x^T(k)R_1x(k) + u^T(k)R_2u(k)) + J(x(k+1)), \quad R_1 \geq 0,\, R_2 > 0 \end{aligned} \tag{5.12}$$

where $r(k)$ represents the immediate return. $J(x(k)) = V(x(k))$ is the value function of the corresponding reinforcement learning problem formulation. Note that RL formula (5.12) can be recorded as

$$J(x(k)) = V(x(k)) = \frac{1}{2}(x^T(k)R_1x(k) + u^T(k)R_2u(k)) + V(x(k+1)) \tag{5.13}$$

with (5.13) called the *Bellman equation*, [19]. We know that (5.25) has the value equal to

$$J(x(k)) = \frac{1}{2}x^T(k)Px(k) = V(x(k)) \tag{5.14}$$

At $k+1$, we have from (5.14)

$$J(x(k+1)) = \frac{1}{2}x^T(k+1)Px(k+1) = V(x(k+1)) \tag{5.15}$$

Using the linear feedback control $u(k) = -Fx(k)$ in the state equation (5.11) gives $x(k+1) = (A - BF)x(k)$ so that

$$J(x(k+1)) = \frac{1}{2} x^T(k)(A-BF)^T P(A-BF)x(k) \tag{5.16}$$

The linear feedback control $u(k) = -Fx(k)$ used in (5.13) and (5.14) implies that the Bellman equation in the discrete-time LQ optimization satisfies

$$x^T(k)\Big((A-BF)^T P(A-BF) - P + Q + F^T RF\Big)x(k) = 0 \tag{5.17}$$

Since (5.17) must hold for any $x(k)$, it follows that the *Bellman equation of RL in the case of discrete-time LQ optimal control systems is equivalent to the discrete-time algebraic Lyapunov equation*

$$P = (A-BF)^T P(A-BF) + Q + F^T RF \tag{5.18}$$

The optimal feedback control is given by $u^{opt}(x^{opt}(k)) = -F^{opt}x^{opt}(k)$ with the optimal feedback gain

$$F^{opt} = \Big(R + B^T PB\Big)^{-1} B^T PA \tag{5.19}$$

where P is the positive semi-definite stabilizing solution of the well-known *discrete-time Riccati algebraic equation (DARE)* given by

$$P = A^T PA + Q - A^T PB\Big(R + B^T PB\Big)^{-1} B^T PA \tag{5.20}$$

Note that the *Bellman optimality equation of RL in the case of discrete-time LQ optimization is equivalent to the discrete-time algebraic Riccati equation (DARE)*. The positive semidefinite stabilizing solution of DARE is guaranteed under the next assumption.

Assumption 5.1: The pairs (A, B) and $(A, \text{Chol}(Q))$, are respectively controllable and observable or at least stabilizable and detectable.

Using this positive semi-definite stabilizing solution of (5.20) the optimal feedback (closed-loop) system is given by

$$x^{opt}(k+1) = \Big(A - BF^{opt}\Big)x^{opt}(k) \tag{5.21}$$

with the optimal performance criterion evaluated as

$$J^{opt}(x(k)) = \frac{1}{2} x^T(k) P x(k) \tag{5.22}$$

The Hewer algorithm is a policy iterations algorithm, of the general form: $u_i(x(k)) = -F_i x(k),\ i = 0,1,2,\ldots,$ which *starts with some stabilizing* F_0 such that $\left|\lambda_i(A - BF_0)\right| < 1, \forall i$. It iteratively solves a sequence of discrete-time algebraic Lyapunov equations (DALE) , and appropriately updates the approximate feedback gain as following

$$\begin{aligned} P_i &= \left(A - BF_{i-1}\right)^T P_i \left(A - BF_{i-1}\right) + F_{i-1}^T R F_{i-1} + Q, \quad i = 1,2,3,\ldots \\ F_i &= \left(R + B^T P_i B\right)^{-1} B^T P_i A \quad \text{with } F_0 \text{ stabilizing } \quad \left|\lambda_j(A - BF_0)\right| < 1 \end{aligned} \tag{5.23}$$

In this process, in each iterative step the approximate optimal control is improved since the Lyapunov iterations (5.23) have built in the minimization procedure

$$\begin{aligned} \frac{\partial}{\partial u}\left\{J(x(k))\right\} &= \frac{\partial}{\partial u}\left\{\frac{1}{2}\left(x^T(k)Qx(k) + u^T(k)Ru(k)\right) + J(x(k+1))\right\} \\ &= \frac{\partial}{\partial u}\left\{\frac{1}{2}\left(x^T(k)Qx(k) + u^T(k)Ru(k)\right) + J(x(k+1))\right\} \\ &= \frac{\partial}{\partial u}\left\{\frac{1}{2}\left(x^T(k)Qx(k) + u^T(k)Ru(k)\right) + (Ax(k) + Bu(k))^T P_i (Ax(k) + Bu(k))\right\} \end{aligned}$$

which implies

$$Ru_i(k) + B^T P_i B u_i(k) + B^T P_i A x(k) \quad \Rightarrow \quad u_i(x(k)) = -\left(R + B^T P_i B\right)^{-1} B^T P_i A x(k) \tag{5.24}$$

This procedure is repeated leading to

$$\begin{aligned} u_i(x(k)) &= -\left(R + B^T P_i B\right)^{-1} B^T P_i A x(k) = -F_i x(k) \\ J_i(x(k)) &= \frac{1}{2} x^T(k) P_{i+1} x(k), \quad i = 0,1,2,\ldots \\ P_{i+1} &- (A - BF_i)^T P_{i+1} (A - BF_i) = Q + F_i^T R F_i \end{aligned} \tag{5.25}$$

When $u(k) \to u^{opt}(x(k))$ then $P_i \to P$, where P satisfies DARE. The generated sequence of P_i matrices has the property

$$P_1 \geq P_2 \geq P_3 \geq \cdots \geq P \tag{5.26}$$

Where P is the unique stabilizing solution of DARE given in (5.20), and with the approximate performance criterion values converging to the optimal performance criterion value, that is

$$J_0(x(k)) \geq J_1(x(k)) \geq J_2(x(k)) \geq \cdots \geq J_{opt}(x(k)) \tag{5.27}$$

It was stated in [24] that this algorithm has a quadratic rate of convergence.

Comment 5.2: This procedure is equivalent to successive minimization of the Hamiltonian. Given the Hamiltonian and an initial approximate feedback control $u_0(x(k)) = -F_0 x(k)$, we proceed first with an approximate evaluation of the costate variable $p(k+1)$, and then with minimization

$$\begin{aligned} &\min_{u(x(k))} \left\{ H(x(k), u(k), p(k+1) \right\} \\ &= \min_{u(x(k))} \left\{ \frac{1}{2}\left(x^T(k) Q x(k) + u^T(k) R u(k) \right) + p(k+1)\left(Ax(k) + Bu(k) \right) \right\} \end{aligned} \tag{5.28}$$

producing new approximate control $u_1(x(k))$, and iterative repeat the process until $u_i(x(k)) \to u_{opt}(x(k))$.

5.3 Value Iterations in Discrete Time

Recall that for a general time-invariant dynamic optimization problem defined by a nonlinear system and corresponding performance criterion that has to be minimized as

$$\begin{aligned} &x(k+1) = f(x(k), u(k)), \qquad x(k) = \text{given initial condition} \\ &J(x(k)) = \sum_{k}^{k_f} g(x(k), u(k)) + \gamma(x(k_f)) \end{aligned} \tag{5.29}$$

we derived in Module 3, *the Bellman functional equation* in the discrete time

$$J_j^{opt}(x(j)) = \min_{u(j)} \left\{ g(x(j),u(j)) + J_{j+1}^{opt}\left(f(x(j),u(j))\right) \right\},$$
$$j = k_f - k_0, k_f - k_0 - 1, ..., 2, 1, 0, \qquad (.5.30)$$
$$J_{k_f}^{opt}(x(k_f - k_0)) = \text{known} = \gamma(x(k_f))$$

One way to solve Bellman's functional equation in discrete-time for a *time-invariant* nonlinear system and the corresponding performance criterion defined *over infinite time interval* for any initial time k by

$$x(k+1) = f(x(k),u(k)), \qquad x(k) = \text{initial condition is given}$$
$$J(x(k)) = \sum_{k}^{\infty} g(x(k),u(k)) \qquad (5.31)$$

is to perform iterations in the function space by guessing an initial (approximate) value for the performance criterion, say $J^{opt^{(0)}}(x(k))$, and running successive iterations as proposed and demonstrated in [8]-[9], [26]. Observe first that the Bellman functional equation for time-invariant systems can be written, using a slightly different form, which is valid for any discrete-time *k*, as

$$J^{opt}(x(k)) = \min_{u(k)} \left\{ g(x(k),u(k)) + J^{opt}\left(x(k+1)\right) \right\} \qquad (5.32)$$

The following value iterations successive scheme, based on the ideas of [2], was presented in [8]-[9], [26].

Value Iterations Algorithm:

$$J^{opt^{(i+1)}}(x(k)) = \min_{u(x(k))} \left\{ g\left(x(k),u(k)\right) + J^{opt^{(i)}}\left(x(k+1)\right) \right\}$$
$$J^{opt^{(0)}}(x(k)) = \text{initial guess}, \quad i = 0,1,2,... \qquad (5.33)$$

It should be emphasized that there are two iterative processes involved in (5.33). Firstly, for the given initial guess of the performance criterion, the minimization has to be performed to find $u^{(1)}(x(k))$ that minimizes the right-hand side of the functional equation (5.33), that is

$$\begin{aligned}&\frac{\partial}{\partial u}\left\{g\left(x(k),u(k)\right)+J^{opt^{(0)}}\left(x(k+1)\right)\right\}\\&=\frac{\partial}{\partial u}\left\{g\left(x(k),u(k)\right)+J^{opt^{(0)}}\left(f(x(k),u(k))\right)\right\}=0\quad\Rightarrow\quad u^{(1)}(x(k))\end{aligned}\tag{5.34}$$

and secondly, upon obtaining the function $u^{(1)}(x(k))$, assuming it exists, evaluation of the approximate performance criterion $J^{opt^{(1)}}(x(k))$ is done as follows

$$J^{opt^{(1)}}(x(k))=g\left(x(k),u^{(1)}(x(k))\right)+J^{opt^{(0)}}\left(x(k+1)\right)\tag{5.35}$$

Note that for the given approximate optimal control $u^{(1)}(x(k))$ and the corresponding state equation $x(k+1)=f(x(k),u^{(1)}(k))$, the first approximation of the optimal performance criterion is given by

$$J^{opt^{(1)}}(x(k))=g\left(x(k),u^{(1)}(x(k))\right)+J^{opt^{(0)}}\left(f(x(k),u^{(1)}(x(k)))\right)\tag{5.36}$$

which gives the *approximate optimal performance value in terms of the state variables only*. In the next step of algorithm, $J^{opt^{(1)}}(x(k))$ is used to find $J^{opt^{(2)}}(x(k))$ by repeating steps (5.34)-(5.36) with the new initial value and the new approximate performance value to be evaluated, that is

$$\begin{aligned}&\frac{\partial}{\partial u}\left\{g\left(x(k),u(k)\right)+J^{opt^{(1)}}\left(x(k+1)\right)\right\}\\&\quad=\frac{\partial}{\partial u}\left\{g\left(x(k),u(k)\right)+J^{opt^{(1)}}\left(f(x(k),u(k))\right)\right\}=0\quad\Rightarrow\quad u^{(2)}(x(k))\end{aligned}\tag{5.37}$$

and using $u^{(2)}(x(k))$ to evaluate the approximate performance criterion $J^{opt^{(2)}}(x(k))$ as

$$J^{opt^{(2)}}(x(k))=g\left(x(k),u^{(2)}(x(k))\right)+J^{opt^{(1)}}\left(x(k+1)\right)\tag{5.38}$$

Note that for the new approximate control $u^{(2)}(x(k))$ and the corresponding state equation $x(k+1)=f(x(k),u^{(2)}(k))$, the optimal performance criterion approximation is given by

$$J^{opt^{(2)}}(x(k)) = g\left(x(k), u^{(2)}(x(k))\right) + J^{opt^{(1)}}\left(f(x(k), u^{(2)}(x(k)))\right) \tag{5.39}$$

which gives the new *approximate optimal performance value in terms of the state variables only*. This procedure is repeated until the convergence is achieved, leading to the following two stage general iterative scheme, which is equivalent to the original value iterations scheme

$$\begin{aligned} &\frac{\partial}{\partial u}\left\{g\left(x(k), u(k)\right) + J^{opt^{(i)}}\left(f(x(k), u(k)\right)\right\} = 0 \quad \Rightarrow \quad u^{(i+1)}(x(k)) \\ &J^{opt^{(i+1)}}(x(k)) = g\left(x(k), u^{(i+1)}(x(k))\right) + J^{opt^{(i)}}\left(f(x(k), u^{(i+1)}(x(k)))\right) \\ &J^{opt^{(0)}}(x(k)) \text{ given} \end{aligned} \tag{5.40}$$

Note that in fact (5.40) represents the *generalized policy iterations*, the terminology used in Chapter 6 of the book by [23], which represents an interplay of the value function improvements and the policy improvements by going back-and-forth to each other. The simplest way to do this exchange of intermediate values is via the synchronous algorithm as given by (5.40), but it is possible also to derive an asynchronous algorithm version of (5.40). *However, convergence of the value iterations algorithm (5.33) to the optimal performance value and to the optimal feedback control (strategy) is not guaranteed.* According to [26], if the initial guess is good, $J^{opt^{(0)}}(x(k))$, is close to $J^{opt}(x(k))$, the iterative scheme (5.33) may converge to the true value of the performance criterion. In general, the convergence condition might not exist or the conditions for convergence might be very restrictive.

***Comment 5.3*:** It is interesting to observe that in the value iterations procedure no need existed for evaluating intermediate states

$$x^{(i)}(k+1) = f(x^{(i)}(k), u^{(i)}(k)), \qquad x^{(i)}(k) = x(k) \text{ initial condition is given}$$

which was the requirement in continuous-time Vaisbord-Milstein's algorithm.

5.3.1 Value Iterations for Discrete-Time LQ Optimal Control Problem

VIs are developed for the steady state LQ optimal control time-invariant problem defined by (5.11)-(5.12), when the optimization period theoretically goes to infinity. In that case, the optimal performance in the long run takes a constant value, that is, the Bellman functional equation

$$J_k^{opt}(x(k)) = \min_{u(k)}\left\{g(x(k), u(k)) + J_{k+1}^{opt}\left(f(x(k), u(k))\right)\right\}, \quad k = 0, 1, 2, \ldots \tag{5.41}$$

becomes

$$J^{opt}(x(k)) = \min_{u(k)} \left\{ g(x(k), u(k)) + J^{opt}(x(k+1)) \right\}, \quad \text{for any } k \tag{5.42}$$

since at steady state $x(k+1) \to x(k)$ and $J^{opt}(x(k+1)) \to J^{opt}(x(k))$. Equation (5.42) can be used in general to find $J^{opt}(x(k))$ via the *fixed-point iterations algorithm* (*VIs*) as follows

$$\begin{aligned} & J^{opt^{(i+1)}}(x(k)) = \min_{u(k)} \left\{ g(x(k), u(k)) + J^{opt^{(i)}}(x(k+1)) \right\} \\ & \text{for some } J^{opt^{(0)}}(x(k)), \quad i = 0, 1, 2, \ldots \end{aligned} \tag{5.43}$$

The min operator in (5.43) indicates that in addition iterations with respect to the approximate optimal control have to be run. Minimization of the right-hand side of (5.43) implies

$$\begin{aligned} & \frac{\partial}{\partial u(k)} \left\{ g(x(k), u(k)) + J^{opt^{(i)}}(x(k+1)) \right\} = 0 \\ & \Rightarrow \quad u^{(i)}(k) = \arg\min \left\{ g(x(k), u(k)) + J^{opt^{(i)}}(x(k+1)) \right\} \end{aligned} \tag{5.44}$$

For the discrete-time LQ optimization defined by (5.11)-(5.12), we have

$$V(x(k)) = J(x(k)) = \frac{1}{2} x^T(k) P x(k) \quad \Rightarrow \quad J(x(k+1)) = \frac{1}{2} x^T(k+1) P x(k+1)$$

so that

$$\begin{aligned} & \frac{\partial}{\partial u} \left\{ \frac{1}{2} \left(x^T(k) Q x(k) + u^T(k) R u(k) \right) + J^{(i)}(x(k+1)) \right\} \\ & = \frac{\partial}{\partial u} \left\{ \frac{1}{2} \left(x^T(k) Q x(k) + u^T(k) R u(k) \right) + J^{(i)}(x(k+1)) \right\} \\ & \quad = \frac{\partial}{\partial u} \left\{ \frac{1}{2} \left(x^T(k) Q x(k) + u^T(k) R u(k) \right) + (Ax(k) + Bu(k))^T P^{(i)} (Ax(k) + Bu(k)) \right\} = 0 \end{aligned}$$

which implies

$$u_i(x(k)) = -\left(R + B^T P^{(i)} B \right)^{-1} B^T P^{(i)} A x(k) = -F^{(i)} x(k) \tag{5.45}$$

The value function $V(x(k)) = J(x(k))$ satisfies the Bellman equation (5.42), which in view of VIs algorithm (5.40) produces the value iterations algorithm for discrete-time LQ given by

$$\begin{aligned} V^{(i+1)}(x(k)) &= \frac{1}{2}(x^T(k)Qx(k) + u^T(k)Ru(k)) + V^{(i)}(x(k+1)), \quad i = 0,1,2,... \\ V^{(0)} &= \frac{1}{2}x^T(k)P^{(0)}x(k) \text{ given} \end{aligned} \tag{5.46}$$

The Bellman equation has the form

$$x^T(k)Px(k) = x^T(k)(Q + F^T RF)x(k) + x^T(k)(A - BF)^T P(A - BF)x(k)$$

which in view of the value iterations algorithm can be written as

$$x^T(k)P^{(i+1)}x(k) = x^T(k)(Q + F^{T^{(i)}} RF^{(i)})x(k) + x^T(k)(A - BF^{(i)})^T P^{(i)}(A - BF^{(i)})x(k)$$

with the approximate gains obtained from (4.45). Since the last equation is valid for any $x(k)$, the following recursive formula is obtained for evaluation of P_i

$$\begin{aligned} P^{(i+1)} &= (A - BF^{(i)})^T P^{(i)}(A - BF^{(i)}) + Q + F^{T^{(i)}} RF^{(i)}, \quad P^{(0)} \text{ given} \\ F^{(i)} &= \left(R + B^T P^{(i)} B\right)^{-1} B^T P^{(i)} A, \quad i = 0,1,2,... \end{aligned} \tag{5.47}$$

It was stated that the initial condition in (5.47) can be taken as $P^{(0)} = 0$, [27]: "This algorithm does not require that the initial gain be stabilizing and can be initialized with any feedback gain". Similar statement can be found in [19]: "This algorithm is shown to converge in [45] to the solution of the Riccati equation… ". Reference "[45]" is the known book on the algebraic Riccati equation, cited in this document as reference [28].

Comparison of Policy Iterations and Value Iterations

Comparing PIs (5.23) and VIs (5.47), we see that the PIs require solving at each iterative step a DALE and that it requires also an initial stabilizing feedback gain. On the other hand, VIs algorithm (5.47) does not require solving any equation. Instead a recursive formula is run that "… can be initialized with any feedback gain", [19]. The approximate optimal feedback gain and the approximate optimal performance criteria are given exactly by the same formulas as given in (5.25). Both solutions of (5.23) and (5.47) converge to the unique positive semidefinite stabilizing solution of DARE (5.33). It will be interesting to compare the speed of convergence of these two algorithms, and consider differences when we start the value VIs with a stabilizing initial feedback gain and when the initial feedback gain is destabilizing. More about of

comparison of the RL PIs and RL VIs for the LQ optimal discrete-time control problem will be presented Section 5.4 in the light of results obtained in a very recent paper [30].

Comment 5.4: *Why We Need VIs and PIs in Linear-Quadratic Optimization?*
The solution to the linear-quadratic optimization problem based on the Riccati equation imposed no constraints on the state variables and control input. This method does not apply to the case when state variables are constrained to some space, say $x(k) \in X$, and control variables are constrained to another space, say $u(k) \in U$. In practice, we often have $\|u(k)\| \le \text{constant}=\text{U}_1 \in \text{R}^m$, indicating that the system cannot tolerate large input signals at any time, or $\|x(k)\| \le \text{constant}=\text{X}_1 \in \text{R}^n$, indicating that the state variables cannot take arbitrary large values at any time. *The dynamic programming method can handle these constrained cases as well by quantizing the state and control variables with respect to their magnitudes at any given discrete time*. In the case of higher order dimensional systems quantization introduces a tremendous computational burden, but the optimal feedback solution can be obtained (assuming it exists). In addition, *when the optimization period is very long or infinity these iterations can provide either the optimal or near-optimal solutions*.

Example 5.1: In this example, we compare the PIs algorithm that requires solving the DALE (5.23) and the VIs algorithm that requires only function evaluation (5.47). Note that both PIs and VIs have identical formulas for the approximate gains. Consider the following discrete-time model of a steam power system [29], whose LQ optimal control problem matrices are

$$A = \begin{bmatrix} 0.9150 & 0.0510 & 0.0380 & 0.0150 & 0.0380 \\ -0.0300 & 0.8890 & -0.0005 & 0.0460 & 0.1110 \\ -0.0600 & 0.4680 & 0.2470 & 0.0140 & 0.0480 \\ -0.7150 & -0.0220 & 0.0211 & 0.2400 & -0.0240 \\ -0.1480 & -0.0030 & -0.0040 & 0.0900 & 0.0260 \end{bmatrix}, \quad B = \begin{bmatrix} 0.0098 \\ 0.1220 \\ 0.0360 \\ 0.5620 \\ 0.1150 \end{bmatrix}$$

$$R_1 = \text{diag}\{5 \quad 5 \quad 5 \quad 5 \quad 5\}, \quad R_2 = 1, \quad x^T(0) = [1 \quad 1 \quad 1 \quad 1 \quad 1]$$

Starting with the stabilizing initial feedback gain equal to $F^{(0)} = [-1 \quad 1 \quad 1 \quad 1 \quad 5]$, the solution of DARE with the accuracy of four decimal digits and the corresponding approximate and optimal performance values are obtained after five iterations of the PIs algorithm (5.23). These values for the approximate performance criteria are given in Table 5.1.

Iteration number	Approximate performance criteria $J_{appr}^{(i)} = 0.5x^T(0)P^{(i+1)}x(0)$
1	336.3446
2	74.2239
3	56.0299
4	55.4123
5	55.4100 = optimal
6	55.4100

Table 5.1: Approximate performance criteria for PIs (5.23)

The results presented in this table demonstrate fast quadratic convergence from above to the optimal performance value of $J_{opt} = 55.4100$, and hence display the RL mechanism in the sense that reinforcing the strategy $u^{(i)}(x(k)) = -F^{(i)}x(k)$, as defined in (5.25), produces better and better awards with each iteration i (with each repetition of the control action).

Consider now the VIs for the same example. Since the system matrix A is asymptotically stable, we have selected as an initial condition $P^{(0)} = 0$, which implies $F^{(0)} = 0$ so that the closed-loop system $A - BF^{(0)}$ is also asymptotically stable. The VIs algorithm with a stabilizing initial feedback gain converges very slowly as demonstrated in Table 5.2. It took 64 iterations to achieve the optimal value with the accuracy of four decimal digits, compared to 5 iterations that the PIs algorithm needed for the same accuracy.

Iteration number	Approximate performance criterion $J_{appr}^{(i)} = 0.5x^T(0)P^{(i+1)}x(0)$
1	12.5000
2	19.7930
3	26.6630
4	32.4782
5	37.2080
6	40.9890
10	49.7364
20	54.8666
30	55.3588
40	55.4052
50	55.4095
60	55.4099
64	55.4100
65	55.4100

Table 5.2: Approximate performance criteria for VIs (5.47)

We have experimented with unstable initial guesses. Despite It took in some cases more than one hundred iterations to achieve convergence with the accuracy of four decimal digits.

```
clear all
A=[0.9150 0.0510 0.0380 0.0150 0.0380;
    -0.0300 0.8890 -0.0005 0.0460 0.1110;
    -0.0600 0.4680 0.2470 0.0140 0.0480;
    -0.7150 -0.0220 -0.0211 0.2400 -0.0240;
    -0.1480 -0.0030 -0.0040 0.0900 0.0260];
evA=eig(A);
B=[0.0098;0.1220;0.0360;0.5620;0.1150];
x0=[1;1;1;1;1];
R1=5*eye(5); R2=1;
F0=[-1 1 1 1 5];
ev=eig(A-B*F0); absev=abs(ev);
% Direct Optimal Solution
[Fopt,P]=dlqr(A,B,R1,R2);
```

```
Jopt=0.5*x0'*P*x0;
Fi=F0;
% POLICY ITERATIONS
for i=1:10
    Pi=dlyap((A-B*Fi)',R1+Fi'*R2*Fi);
    Fi=inv(R2+B'*Pi*B)*B'*Pi*A;
    Ji(i)=0.5*x0'*Pi*x0;
end
figure (1)
plot(Ji,'*'); grid; xlabel('Iteration'); ylabel('Performance')
% VALUE ITERATIONS
P00=zeros(5,5)
F00=inv(R2+B'*P00*B)*B'*P00*A
for j=1:70
    Pj=(A-B*F00)'*P00*(A-B*F00)+R1+F00'*R2*F00;
    P00=Pj;
    F00=inv(R2+B'*Pj*B)*B'*Pj*A;
    Jj(j)=0.5*x0'*Pj*x0;
end
figure (2)
plot(Jj,'*'); grid; xlabel('Iteration'); ylabel('Performance')
```

5.4 Acceleration of the RL Value Iterations Algorithm

In the previous two sections, we have made interesting observations regarding the policy and value iterations used to solve the discrete-time linear-quadratic optimal control problem. We used simulation to demonstrate and emphasize these observations. In this section, prepared by following a very recent paper [30], we show that convergence of the RL value VIs algorithm of the discrete-time linear-quadratic (LQ) optimal control problem can be made quadratic by using the doubling algorithm, which represents an improvement over the previously known linear convergence. The section also establishes a milder condition (see Assumption 5.2 stated later in this section) needed for convergence of the considered RL VIs algorithm compared to the required convergence condition given in Assumption 5.1 for the RL PIs algorithm of the discrete-time LQ optimal control problem. In addition, the paper [30] shows in the that the presented VIs algorithm is numerically much more efficient than the corresponding PIs algorithm since it requires much smaller number of flops per iteration, that is, $17n^3$ vs $30n^3$ needed for the PIs algorithm.

It can be shown after some algebra, by eliminating the gain $F^{(i)}$ from the first iterative formula in (5.47) using the second iterative formula in (5.47), that the VIs algorithm (5.47) can

be represented by an independent iterative scheme with respect to $P^{(i)}$ that does not involve the approximate optimal feedback gain (action):

$$\begin{aligned} P^{(i+1)} &= A^T P^{(i)} A - A^T P^{(i)} B(R + B^T P^{(i)} B)^{-1} B^T P^{(i)} A + Q, \quad P^{(0)} \text{ given} \\ F^{(i)} &= \left(R + B^T P^{(i)} B\right)^{-1} B^T P^{(i)} A, \quad i = 0,1,2,... \end{aligned} \tag{5.48}$$

The iterations for $P^{(i)}$, in fact represent the difference Riccati equation whose limiting form, assuming its solution in the limit exists when $i \to \infty$ is the discrete-time algebraic Riccati equation defined in (5.20). When iterated forward in time from some symmetric initial condition $P^{(0)}$, this equation corresponds to the difference Riccati equation of the optimal Kalman filter [31]. Several authors have been studied the convergence properties of the difference Riccati equation (5.48) to the corresponding algebraic Riccati equation (5.20), for example, [32]-[34]. It was concluded that under Assumption 5.1 and $P^{(0)} = 0$, the convergence of $P^{(i)} \to P$ is monotonic, in which case $P^{(i)}$ represents a non-decreasing sequence of positive semi-definite matrices with the feedback matrix $A - BF^{opt}$ being asymptotically stable, [29]. In the case when the initial condition is any positive semidefinite matrix, that is, $P^{(0)} \geq 0$ only non-monotonic convergence is achieved under Assumption 5.1 with $A - BF^{opt}$ being also asymptotically stable, [28]. The paper [34] provides a convergence result of the limiting solution of (5.48) to the stable (though not asymptotically stable) solution of (5.20) under a very mild condition.

Assumption 5.2: The pair (A, B) is stabilizable.

The solution obtained under Assumption 5.2 is so-called "strong solution" [34] with the eigenvalues of feedback matrix $A - BF^{opt}$ being either inside of the unit circle or on the unit circle and preserving the closed-loop system stability. Hence, *the value iteration algorithm can be implemented under a weaker condition as given in Assumption 5.2 than the policy iteration algorithm that requires Assumption 5.1.* Assumption 5.1 is needed to provide the asymptotic stability of the feedback matrix $A - BF^{opt}$ and it is also needed to assure the existence of unique solutions of the algebraic Lyapunov equations in each iteration of (5.23), [24]-[25].

The slow linear convergence of (5.48) can be made quadratic by using the doubling idea of Smith [35] based on the requirement that a sequence

$$P^{(0)}, P^{(1)}, P^{(2)}, P^{(4)}, P^{(8)}, P^{(16)}, ... \tag{5.49}$$

can be generated. The doubling algorithm for solving the considered difference Riccati equation can be found in [36]-[37]. It is given by the following simultaneous set of three recursive formulas

$$M^{(i+1)} = M^{(i)}(I + N^{(i)}V^{(i)})^{-1}M^{(i)}, \quad M^{(0)} = A \tag{5.50a}$$

$$N^{(i+1)} = N^{(i)} + M^{(i)}(I + N^{(i)}V^{(i)})^{-1}N^{(i)}M^{(i)^T}, \quad N^{(0)} = BR^{-1}B^T \tag{5.50b}$$

$$V^{(i+1)} = V^{(i)} + M^{(i)^T}V^{(i)}(I + N^{(i)}V^{(i)})^{-1}M^{(i)}, \quad V^{(0)} = Q \tag{5.50c}$$

with

$$V^{(i)} = P^{(2^i)}, \quad i = 1, 2, ... \tag{5.51}$$

$$F^{(i)} = (R + B^T V^{(i)} B)^{-1} B^T V^{(i)} A, \quad i = 0, 1, 2, ... \tag{5.52}$$

It was shown in [35] that the required matrix inversion (identical in all three equations (5.50)) exists in each iteration.

Example 5.2: We solve the same problem as in Example 5.1 now using the doubling algorithm (5.50)-(5.52) of the original VIs algorithm. The obtained results are presented in Table 5.2. It can be seen from this table that the optimal solution has been obtained in 7 iterations. In Example 5.1, where the original value iteration algorithm is presented, it took 64 iterations to achieve the same optimal value.

The number of flops per iteration is counted in MATLAB while solving Example 5.2 with the doubling algorithm. The total number of flops for the doubling algorithm is equal 2133 per iteration, which is about $17n^3 = 2125$ flops per iteration. The MATLAB flop operation counts for solving DALE is 3905 flops. With additional 167 flops for obtaining the gain $F^{(i)}$, the PIs requires 4072 flops per iteration, which is consistent with the theoretical estimate of $30n^3 = 3750$ flops per iteration.

Iteration number	Approximate performance criterion $J_{appr}^{(i)} = 0.5x^T(0)P^{(2^i)}x(0)$
1	4.1218×10^{1}
2	4.9431×10^{2}
3	2.3902×10^{4}
4	9.7157×10^{5}
5	3.2116×10^{6}
6	3.5258×10^{6}
7	$3.5464\times10^{6} = J^{opt}$

Table 5.2: Approximate performance criteria for PIs doubling algorithm (5.80)-(5.82)

Note in the considered LQ learning process based either on PIs or VIs, the approximate feedback gain is learned, and there is no need to implement the approximate feedback controls (RL actions) in each iteration as $u_{appr}^{(i)}(x_{appr}^{(i)}(t)) = -F_{appr}^{(i)}x_{appr}^{(i)}(t)$. Rather, when the optimal feedback gain is obtained with the desired accuracy, it should be implemented to the system as $u^{opt}(x^{opt}(t)) = -F^{opt}x^{opt}(t)$.

The following MATLAB script implements the VI doubling algorithm for the discrete-time optimal control problem considered in Example 5.2.

```
%% Example 5_2 MATLAB script
clear all
A=[0.9150 0.0510 0.0380 0.0150 0.0380;
    -0.0300 0.8890 -0.0005 0.0460 0.1110;
    -0.0600 0.4680 0.2470 0.0140 0.0480;
    -0.7150 -0.0220 -0.0211 0.2400 -0.0240;
    -0.1480 -0.0030 -0.0040 0.0900 0.0260];
evA=eig(A);
B=[0.0098;0.1220;0.0360;0.5620;0.1150];
x0=[1;1;1;1;1];
R1=5*eye(5); R2=1;
% Direct Optimal Solution via DARE
[Fopt,P]=dlqr(A,B,R1,R2);
Jopt=0.5*x0'*P*x0
%
% VALUE ITERATIONS via the  Doubling Algorithm
%
M0=A;
```

```
N0=B*inv(R2)*B';
V0=R1;
for i=1:10
    W0=inv(eye(5)+N0*V0);
    Mi=M0*W0*M0;
    Ni=N0+M0*W0*N0*M0';
    Vi=V0+M0'*V0*W0*M0;
    M0=Mi;
    N0=Ni;
    V0=Vi;
    i
    Ji(i)=0.5*x0'*Vi*x0
    error=norm(Vi-A'*Vi*A+A'*Vi*B*inv(R2+B'*Vi*B)*B'*Vi*A,2);
    Error=norm(P-Vi,2)
end
figure (1)
plot(Ji,'*'); grid; xlabel('Iteration'); ylabel('Performance')
```

5.5 References

[1] R. Bellman, "Monotone approximation in dynamic programming and the calculus of variations", *Proceedings of the National Academy of Science*, Vol. 40, 1073-1075, 1954c.

[2] R. Bellman, *Dynamic Programming*, Princeton University Press, Princeton, NJ, 1957.

[3] R. Bellman, *Adaptive Control Processes: A Guider Tour*, Princeton University Press, NJ, 1961.

[4] R. Bellman and S. Dreyfus, *Applied Dynamic Programming*, Princeton University Press, Princeton, NJ 1962.

[5] E. Vaisbord, "An approximate method for the synthesis of optimal control," *Automation and Remote Control*, Vol. 24, 1626-1632, 1963.

[6] G. Milhstein, "Successive approximations for solution of one optimal problem," *Automation and Remote Control*, Vol. 25, 321-329, 1964.

[7] R. Leake and R-W. Liu, "Construction of suboptimal control sequences," *SIAM Journal on Control*, Vol. 5, 54-63, 1967.

[8] R. Larson, "A survey of dynamic programming computational procedures," *IEEE Transactions on Automatic Control*, Vol. AC-12, 767-774, 1967.

[9] R. Larson, *State Increment Dynamic Programming*, Elsevier, New York, 1968.

[10] G. Saridis and C-S. Lee, "An approximation theory of optimal control for trainable manipulators," *IEEE Transactions on Systems, Man, and Cybernetics*, Vol. SMC-9, 152-159, 1979.

[11] R. Beard, G. Saridis, and J. Wen, "Approximate solutions of the time-invariant Hamilton-Jacobi-Bellman equation," *Journal of Optimization Theory*, Vol. 96, 589-626, 1998.

[12] D. Bertsekas and J. Tsitsikls, *Neuro-Dynamic Programming*, Athena Scientific, Nahua, NH, 1996.

[13] J. Murray, C. Cox, G. Lendaris, and R. Saeks," Adaptive dynamic programming," *IEEE Transactions on Systems, Man, and Cybernetics- Part C: Applications and Reviews*, Vol. 32, 140-153, 2002.

[14] D. Liu and Q. Wei, "Policy iteration dynamic programming algorithm for discrete-time nonlinear systems," *IEEE Transactions on Neural Networks and Learning Systems*," Vol. 25, 621-634, 2014.
[15] D. Bertsekas, "Value and policy iterations in optimal control and adaptive dynamic programming," *IEEE Transactions on Neural Networks and Learning Systems*," Vol. 28, 500-509, 2017.
[16] D. Bertsekas, *Dynamic Programming and Optimal Control: Approximate Dynamic Programming*, Athina Scientific, Belmont, MA, 2012.
[17] D. Bertsekas, *Reinforcement Learning and Optimal Control*, Athina Scientific, Belmont, MA, 2019.
[18] F. Lewis and D. Vrabie, "Reinforcement learning and feedback control," *IEEE Circuits and Systems Magazine*, 32-50, Third Quarter, 2009.
[19] F. Lewis, D. Vrabie, and K. Vamvoudakis, "Reinforcement learning and feedback control," *IEEE Control Systems Magazine*, 76-105, Dec. 2012.
[20] B. Kiumarsi, K. Vamvoudakis, H. Modares, and F. Lewis, "Optimal and autonomous control using reinforcement learning: A survey," *IEEE Transactions on Neural Networks and Learning Systems*, Vol. 29, 2042-2062, 2018.
[21] Wang, X., S. Wang, X. Liang, D Zhao, J. Huang, and X. Xu, "Deep reinforcement learning: A Survey," *IEEE Transactions on Neural Networks and Learning Systems*, Vol. 35, 5064-5078, 2024.
[22] Kleinman, D., "On an iterative technique for Riccati equations computations," *IEEE Transactions on Automatic Control*," Vol. 13, 114-115, 1968.
[23] R. Sutton and A. Barto, *Reinforcement Learning: An Introduction*, MIT Press, Cambridge, MA, 2020.
[24] G. Hewer, "An iterative technique for the computation of the steady state gains for the discrete optimal regulator," *IEEE Transactions on Automatic Control*, Vol. AC-16, 382-384, 1971.
[25] G. Hewer, "Analysis of a discrete matrix Riccati equation of linear control and Kalman filter, *Journal of Mathematical Analysis and Applications*, Vol. 42, 226-236, 1973.
[26] R. Larson and J. Casti, *Principles of Dynamic Programming: Part I Basic Analytic and Computational Methods*, Marcel Dekker, 1978.
[27] A. Al-Tamimi, F. Lewis, and M. Abu-Khalif, "Discrete-time nonlinear HJB solution using approximate dynamic programming: Convergence proofs," *IEEE Transactions on Systems, Man, and Cybernetics-Part B: Cybernetics*, Vol. 38, 943-949, 2008.
[28] P. Lancaster and L. Rodman, *Algebraic Riccati Equations*, Oxford University Press, 1995.
[29] Z. Gajic and M. Lim, *Optimal Control of Singularly Perturbed Linear Systems and Applications: High Accuracy Techniques*, Marcel Dekker, New York, 2001.
[30] L. Xu and Z. Gajic, "Improving reinforcement learning value iterations in discrete-time linear-quadratic optimal control," *ASME Journal of Dynamic Systems, Measurement and Control*, Vol. 148, 0110081 – 011081-8, 2026.
[31] B. Anderson and J. Moore, *Optimal Filtering*, Prentice Hall 1979.
[32] P. Caines and D. Mayne, "On the discrete time matrix Riccati equation of optimal control," International Journal of Control, Vol. 12, 785-794, 1970.
[33] H. Wimmer, "Existence of positive-definite and semidefinite solutions of discrete-time algebraic Riccati equations, *International Journal of Control*, Vol. 59, 463-471, 1994.

[34] R. Bitmead, M. Gevers, I. Petersen, and R. Kaye, "Monotonicity and stabilizability properties of solutions of the Riccati difference equation: Propositions, lemmas, theorems, fallacious conjectures and counter examples," *Systems & Control Letters*, Vol. 5, 309-315, 1985.

[35] R. Smith, "Matrix equation $XA+BX=C$," *SIAM Journal on Applied Mathematics*, vol. 16, 198-201, 1968.

[36] B. Anderson, "Second-order convergent algorithms for the steady state Riccati equation," *International Journal of Control*, Vol. 28, 295-306, 1978.

[37] M. Kimura, "Convergence of the doubling algorithm for the discrete-time algebraic Riccati equation," *International Journal of Systems Science*, Vol. 19, 701-711, 1988.